

Практическая работа 36. Views в Compose

Узнайте, как добавлять и использовать существующие представления в приложении, созданном с помощью Jetpack Compose.

Взаимодействие с представлениями в Compose

Узнайте, как добавлять представления и использовать API-интерфейсы совместимости в коде Compose.

На данном этапе курса вы хорошо освоили создание приложений с помощью Compose и имеете некоторые знания о создании приложений с помощью XML, Views, View Bindings и Fragments. После создания приложений с представлениями вы, возможно, оценили удобство создания приложений с декларативным пользовательским интерфейсом, таким как Compose. Однако в некоторых случаях имеет смысл использовать Views вместо Compose. В этом коделабе вы узнаете, как использовать View Interops для добавления компонентов View в современное приложение Compose.

На момент написания этого коделаба компоненты пользовательского интерфейса, которые вы собираетесь создать, еще не доступны в Compose. Это прекрасная возможность использовать View Interop!

Предварительные требования:

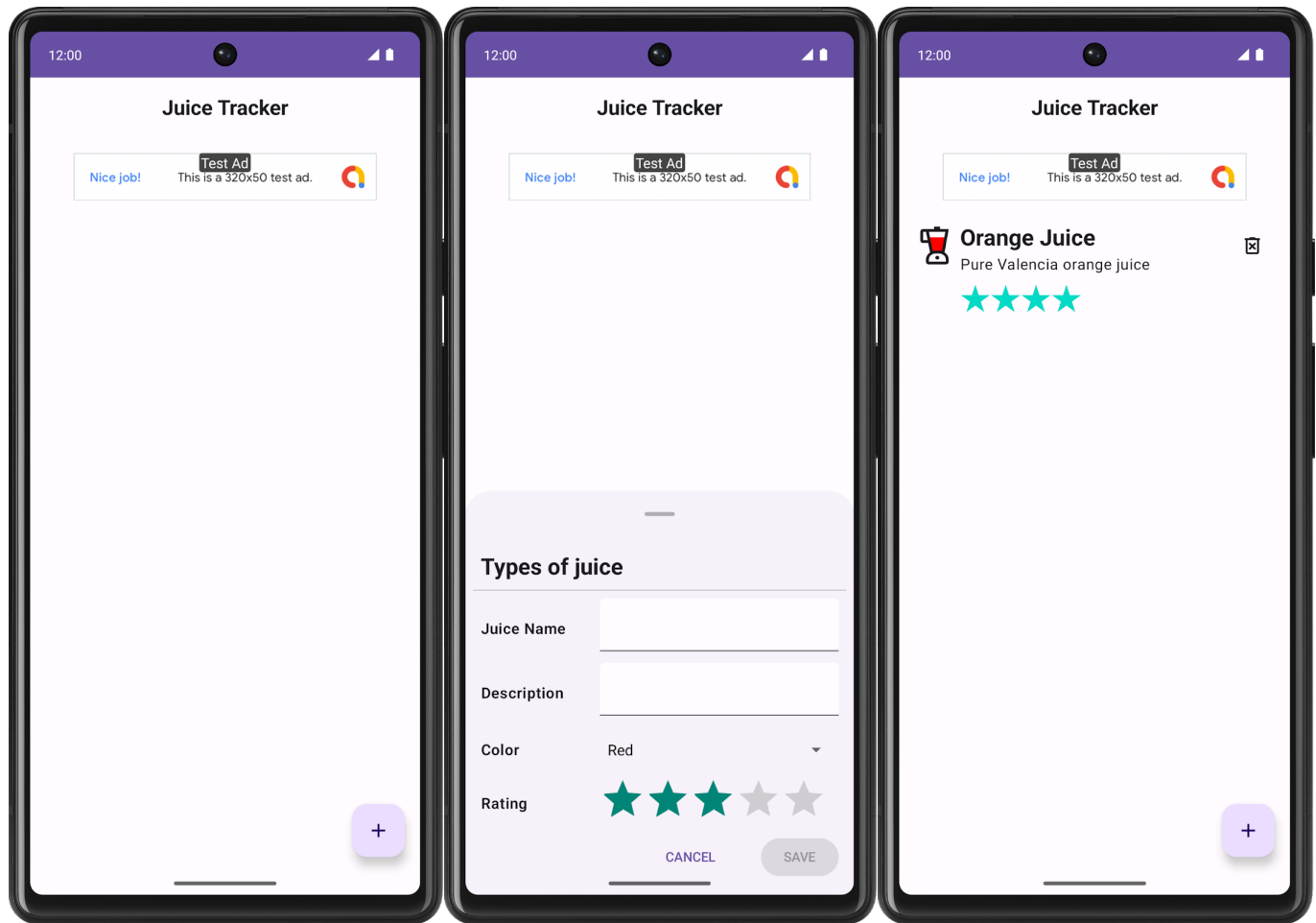
- Пройдите курс «Основы Android с Compose» в рамках урока «Создание Android-приложений с использованием представлений».

Что вам понадобится

- Компьютер с доступом в Интернет и Android Studio
- Устройство или эмулятор
- Стартовый код приложения **Juice Tracker**.

Что вы будете создавать

В этом уроке вам нужно будет интегрировать три представления в пользовательский интерфейс Compose, чтобы завершить пользовательский интерфейс приложения **Juice Tracker**: Spinner, RatingBar и AdView. Для создания этих компонентов вы будете использовать View Interoperability, или сокращенно View Interop. С помощью View Interop вы можете добавить представления в свое приложение, обернув их в Composable.



Прохождение кода В этом мастер-классе вы работаете с тем же приложением JuiceTracker из мастер-классов Build an Android App with Views и Add Compose to a View-based app. Разница в этой версии заключается в том, что предоставленный стартовый код полностью на Compose. В настоящее время в приложении не хватает ввода цвета и рейтинга в диалоговом окне ввода и рекламного баннера в верхней части экрана списка.

Каталог bottomsheet содержит все компоненты пользовательского интерфейса, связанные с диалогом ввода. Этот пакет должен содержать компоненты пользовательского интерфейса для ввода цвета и рейтинга, когда они будут созданы.

Домашний экран содержит компоненты пользовательского интерфейса, размещенные на главном экране, включая список JuiceTracker. Этот пакет должен в конечном итоге содержать рекламный баннер, когда он будет создан.

Основные компоненты пользовательского интерфейса, такие как нижний лист и список соков, размещены в файле JuiceTrackerApp.kt.

2. Получите стартовый код

Чтобы начать работу, загрузите стартовый код:

[file_downloadDownload zip](#)

Также вы можете клонировать репозиторий GitHub для кода:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-juice-tracker.git
$ cd basic-android-kotlin-compose-training-juice-tracker
$ git checkout compose-starter
```

3. Конфигурация Gradle

Добавьте зависимость play services ads в файл build.gradle.kts приложения.

```
app/build.gradle.kts `kt android { ... зависимости { ... implementation(«com.google.android.gms:play-services-ads:22.2.0») } }
```

4. Настройка

Добавьте следующее значение в манифест Android, над тегом activity, чтобы включить рекламный баннер для тестирования:

AndroidManifest.xml

```
```.xml
...
<meta-data
 android:name="com.google.android.gms.ads.APPLICATION_ID"
 android:value="ca-app-pub-3940256099942544~3347511713" />
...

```

Примечание: Значение id, использованное выше, предназначено только для тестирования. Если вы хотите показывать реальные объявления в своем приложении, смотрите раздел Настройка приложения в аккаунте AdMob.

## 5. Заполните диалог ввода

---

В этом разделе вы завершите диалог ввода, создав цветовой спиннер и строку рейтинга. Цветовой волчок - это компонент, позволяющий выбрать цвет, а полоса рейтинга - выбрать оценку для сока. Смотрите дизайн ниже:

Types of juice

Juice Name

Description

Color

Rating

Red

Blue

Green

Cyan

Yellow

Magenta

Orange

Red

★

★

★

★

★

CANCEL

SAVE

{style="width:400px"}

## Rating



```
{style="width:400px"}
```

Создание цветового спиннера Чтобы реализовать спиннер в Compose, необходимо использовать класс `Spinner`. `Spinner` - это компонент `View`, а не `Composable`, поэтому он должен быть реализован с помощью `interop`.

В директории `bottomsheet` создайте новый файл `ColorSpinnerRow.kt`. Внутри файла создайте новый класс `SpinnerAdapter`. В конструкторе `SpinnerAdapter` определите параметр обратного вызова `onColorChange`, который принимает параметр `Int`. `SpinnerAdapter` обрабатывает функции обратного вызова для спиннера.

`bottomsheet/ColorSpinnerRow.kt`

```
class SpinnerAdapter(val onColorChange: (Int) -> Unit){
}
```

Реализуйте интерфейс `AdapterView.OnItemSelectedListener`. Реализация этого интерфейса позволяет определить поведение щелчка для спиннера. Позже вы настроите этот адаптер в `Composable`.

`bottomsheet/ColorSpinnerRow.kt`

```
class SpinnerAdapter(val onColorChange: (Int) -> Unit):
 AdapterView.OnItemSelectedListener {
}
```

Реализуйте функции-члены `AdapterView.OnItemSelectedListener`: `onItemSelected()` и `onNothingSelected()`.

`bottomsheet/ColorSpinnerRow.kt`

```
class SpinnerAdapter(val onColorChange: (Int) -> Unit):
 AdapterView.OnItemSelectedListener {
 override fun onItemSelected(parent: AdapterView<*>?, view: View?, position:
 Int, id: Long) {
 TODO("Not yet implemented")
 }

 override fun onNothingSelected(parent: AdapterView<*>?) {
 TODO("Not yet implemented")
 }
}
```

Измените функцию `onItemSelected()` для вызова функции обратного вызова `onColorChange()`, чтобы при выборе цвета приложение обновляло выбранное значение в пользовательском интерфейсе.

bottomsheet/ColorSpinnerRow.kt

```
class SpinnerAdapter(val onColorChange: (Int) -> Unit):
 AdapterView.OnItemSelectedListener {
 override fun onItemSelected(parent: AdapterView<*>?, view: View?, position:
Int, id: Long) {
 onColorChange(position)
 }

 override fun onNothingSelected(parent: AdapterView<*>?) {
 TODO("Not yet implemented")
 }
 }
}
```

Измените функцию `onNothingSelected()`, чтобы установить цвет на 0, так чтобы при выборе ничего не было, по умолчанию использовался первый цвет, красный.

bottomsheet/ColorSpinnerRow.kt

```
class SpinnerAdapter(val onColorChange: (Int) -> Unit):
 AdapterView.OnItemSelectedListener {
 override fun onItemSelected(parent: AdapterView<*>?, view: View?, position:
Int, id: Long) {
 onColorChange(position)
 }

 override fun onNothingSelected(parent: AdapterView<*>?) {
 onColorChange(0)
 }
 }
}
```

`SpinnerAdapter`, определяющий поведение спиннера с помощью функций обратного вызова, уже создан. Теперь вам нужно создать содержимое спиннера и заполнить его данными.

Внутри файла `ColorSpinnerRow.kt`, но вне класса `SpinnerAdapter`, создайте новый `Composable` под названием `ColorSpinnerRow`. В сигнатуру метода `ColorSpinnerRow()` добавьте параметр `Int` для позиции спиннера, функцию обратного вызова, которая принимает параметр `Int` и модификатор.

bottomsheet/ColorSpinnerRow.kt

```
...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
```

```

 modifier: Modifier = Modifier
) {
 }

```

Внутри функции создайте массив строковых ресурсов цвета сока, используя перечисление `JuiceColor`. Этот массив служит в качестве содержимого, которое будет заполнять спиннер.

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 val juiceColorArray =
 JuiceColor.values().map { juiceColor -> stringResource(juiceColor.label) }
}

```

Добавьте составной `InputRow()` и передайте ресурс `color string` для метки ввода и модификатор, определяющий строку ввода, в которой появится спиннер.

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 val juiceColorArray =
 JuiceColor.values().map { juiceColor -> stringResource(juiceColor.label) }
 InputRow(inputLabel = stringResource(R.string.color), modifier = modifier) {
 }
}

```

Далее вы создадите спиннер! Поскольку `Spinner` - это класс `View`, необходимо использовать API взаимодействия с `View` в `Compose`, чтобы обернуть его в `Composable`. Для этого используется `AndroidView Composable`.

Чтобы использовать спиннер в `Compose`, создайте `AndroidView() Composable` в теле лямбды `InputRow`. `AndroidView() Composable` создает элемент или иерархию `View` в `Composable`.

bottomsheet/ColorSpinnerRow.kt

```
...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 val juiceColorArray =
 JuiceColor.values().map { juiceColor -> stringResource(juiceColor.label) }
 InputRow(inputLabel = stringResource(R.string.color), modifier = modifier) {
 AndroidView()
 }
}
```

AndroidView Composable принимает три параметра:

Лямбда фабрики, которая является функцией, создающей представление. Обратный вызов обновления, который вызывается, когда View, созданный на фабрике, раздувается. Модификатор Composable.

The diagram illustrates the structure of the `AndroidView` Composable. It shows the following code snippet with annotations:

```

AndroidView(
 modifier = Modifier,
 factory = { context ->
 // The View is created here
 },
 update = { view ->
 // The inflated View is accessible here
 }
)

```

Annotations with arrows pointing to the code:

- AndroidView Composable** points to the `AndroidView` function name.
- Composable modifier** points to the `modifier = Modifier` parameter.
- factory lambda** points to the `factory = { context ->` parameter.
- update callback** points to the `update = { view ->` parameter.

Чтобы реализовать `AndroidView`, начните с передачи модификатора и заполнения максимальной ширины экрана. Передайте лямбду для параметра `factory`. Лямбда фабрики принимает в качестве параметра `Context`. Создайте класс `Spinner` и передайте ему контекст.

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 ...
 InputRow(...) {
 AndroidView(
 modifier = Modifier.fillMaxWidth(),

```

```

 factory = { context ->
 Spinner(context)
 }
)
}
}

```

Как RecyclerView.Adapter предоставляет данные для RecyclerView, так и ArrayAdapter предоставляет данные для спиннера. Спиннеру требуется адаптер для хранения массива цветов.

Установите адаптер с помощью ArrayAdapter. ArrayAdapter требует контекст, XML-макет и массив. Передайте simple\_spinner\_dropdown\_item для макета; этот макет предоставляется по умолчанию в Android.

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 ...
 InputRow(...) {
 AndroidView(
 modifier = Modifier.fillMaxWidth(),
 factory = { context ->
 Spinner(context).apply {
 adapter =
 ArrayAdapter(
 context,
 android.R.layout.simple_spinner_dropdown_item,
 juiceColorArray
)
 }
 }
)
 }
}
}

```

Обратный вызов фабрики возвращает экземпляр созданного в ней представления. update - это обратный вызов, который принимает параметр того же типа, что и возвращаемый обратным вызовом фабрики. Этот параметр представляет собой экземпляр View, который был создан фабрикой. В данном случае, поскольку в фабрике был создан спиннер, экземпляр этого спиннера доступен в теле лямбды update.

Добавьте обратный вызов обновления, передающий спиннер. Используйте обратный вызов, переданный в update, для вызова метода setSelection().

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 ...
 InputRow(...) {
 //...
 },
 update = { spinner ->
 spinner.setSelection(colorSpinnerPosition)
 spinner.onItemSelectedListener = SpinnerAdapter(onColorChange)
 }
)
}
}

```

Используйте созданный ранее `SpinnerAdapter`, чтобы установить обратный вызов `onItemSelectedListener()` в обновлении.

bottomsheet/ColorSpinnerRow.kt

```

...
@Composable
fun ColorSpinnerRow(
 colorSpinnerPosition: Int,
 onColorChange: (Int) -> Unit,
 modifier: Modifier = Modifier
) {
 ...
 InputRow(...) {
 AndroidView(
 // ...
),
 update = { spinner ->
 spinner.setSelection(colorSpinnerPosition)
 spinner.onItemSelectedListener = SpinnerAdapter(onColorChange)
 }
 }
)
}
}

```

Код компонента цветового спиннера завершен.

Добавьте следующую служебную функцию, чтобы получить индекс перечисления JuiceColor. Вы будете использовать ее в следующем шаге.

```
private fun findColorIndex(color: String): Int {
 val juiceColor = JuiceColor.valueOf(color)
 return JuiceColor.values().indexOf(juiceColor)
}
```

Реализуйте ColorSpinnerRow в SheetForm Composable в файле EntryBottomSheet.kt. Поместите цветовой спиннер после текста «Описание» и над кнопками.

bottomsheet/EntryBottomSheet.kt

```
...
@Composable
fun SheetForm(
 juice: Juice,
 onUpdateJuice: (Juice) -> Unit,
 onCancel: () -> Unit,
 onSubmit: () -> Unit,
 modifier: Modifier = Modifier,
) {
 ...
 TextInputRow(
 inputLabel = stringResource(R.string.juice_description),
 fieldValue = juice.description,
 onChange = { description -> onUpdateJuice(juice.copy(description
= description)) },
 modifier = Modifier.fillMaxWidth()
)
 ColorSpinnerRow(
 colorSpinnerPosition = findColorIndex(juice.color),
 onChange = { color ->
 onUpdateJuice(juice.copy(color = JuiceColor.values()[color].name))
 }
)
 ButtonRow(
 modifier = Modifier
 .align(Alignment.End)
 .padding(bottom = dimensionResource(R.dimen.padding_medium)),
 onCancel = onCancel,
 onSubmit = onSubmit,
 submitButtonEnabled = juice.name.isNotEmpty()
)
}
}
```

Создайте входные данные рейтинга Создайте новый файл в каталоге bottomsheet под названием RatingInputRow.kt. В файле RatingInputRow.kt создайте новый Composable под названием

RatingInputRow(). В сигнатуре метода передайте Int для рейтинга, обратный вызов с параметром Int для обработки изменения выбора и модификатор.

bottomsheet/RatingInputRow.kt

```
@Composable
fun RatingInputRow(rating: Int, onRatingChange: (Int) -> Unit, modifier: Modifier =
Modifier){
}
```

Как и в случае с ColorSpinnerRow, добавьте в Composable строку InputRow, содержащую AndroidView, как показано в следующем примере.

bottomsheet/RatingInputRow.kt

```
@Composable
fun RatingInputRow(rating: Int, onRatingChange: (Int) -> Unit, modifier: Modifier =
Modifier){
 InputRow(inputLabel = stringResource(R.string.rating), modifier = modifier) {
 AndroidView(
 factory = {},
 update = {}
)
 }
}
```

В теле лямбда-фабрики создайте экземпляр класса RatingBar, который обеспечивает тип рейтинговой панели, необходимый для данного дизайна. Установите значение stepSize равным 1f, чтобы рейтинг был только целым числом.

bottomsheet/RatingInputRow.kt

```
@Composable
fun RatingInputRow(rating: Int, onRatingChange: (Int) -> Unit, modifier: Modifier =
Modifier){
 InputRow(inputLabel = stringResource(R.string.rating), modifier = modifier) {
 AndroidView(
 factory = { context ->
 RatingBar(context).apply {
 stepSize = 1f
 }
 },
 update = {}
)
 }
}
```

Когда представление раздувается, устанавливается рейтинг. Напомним, что фабрика возвращает экземпляр RatingBar в обратный вызов update.

Используйте рейтинг, переданный в Composable, чтобы установить рейтинг для экземпляра RatingBar в теле лямбды update. Когда будет установлен новый рейтинг, используйте обратный вызов RatingBar для вызова функции onRatingChange(), чтобы обновить рейтинг в пользовательском интерфейсе.

bottomsheet/RatingInputRow.kt

```
@Composable
fun RatingInputRow(rating: Int, onRatingChange: (Int) -> Unit, modifier: Modifier =
Modifier){
 InputRow(inputLabel = stringResource(R.string.rating), modifier = modifier) {
 AndroidView(
 factory = { context ->
 RatingBar(context).apply {
 stepSize = 1f
 }
 },
 update = { ratingBar ->
 ratingBar.rating = rating.toFloat()
 ratingBar.setOnRatingBarChangeListener { _, _, _ ->
 onRatingChange(ratingBar.rating.toInt())
 }
 }
)
 }
}
```

Теперь композит ввода рейтинга завершен.

Используйте композит RatingInputRow() в листе EntryBottomSheet. Поместите его после цветового волчка и над кнопками.

bottomsheet/EntryBottomSheet.kt

```
@Composable
fun SheetForm(
 juice: Juice,
 onUpdateJuice: (Juice) -> Unit,
 onCancel: () -> Unit,
 onSubmit: () -> Unit,
 modifier: Modifier = Modifier,
) {
 Column(
 modifier = modifier,
 verticalArrangement = Arrangement.spacedBy(4.dp)
) {
 ...
 ColorSpinnerRow(
```

```

 colorSpinnerPosition = findColorIndex(juice.color),
 onChange = { color ->
 onUpdateJuice(juice.copy(color = JuiceColor.values()[color].name))
 }
)
 RatingInputRow(
 rating = juice.rating,
 onChange = { rating -> onUpdateJuice(juice.copy(rating =
rating)) }
)
 ButtonRow(
 modifier = Modifier.align(Alignment.CenterHorizontally),
 onCancel = onCancel,
 onSubmit = onSubmit,
 submitButtonEnabled = juice.name.isNotEmpty()
)
}
}

```

Создайте рекламный баннер В пакете homescreen создайте новый файл AdBanner.kt. В файле AdBanner.kt создайте новый Composable под названием AdBanner(). В отличие от предыдущих композиций, которые вы создали, AdBanner не требует ввода. Поэтому вам не нужно оборачивать его в InputRow Composable. Однако он требует наличия AndroidView.

Попробуйте создать баннер самостоятельно, используя класс AdView. Убедитесь, что размер объявления установлен в AdSize.BANNER, а идентификатор рекламного блока - в «ca-app-pub-3940256099942544/6300978111». Когда AdView раздуется, загрузите объявление с помощью конструктора AdRequest Builder.

homescreen/AdBanner.kt

```

@Composable
fun AdBanner(modifier: Modifier = Modifier) {
 AndroidView(
 modifier = modifier,
 factory = { context ->
 AdView(context).apply {
 setAdSize(AdSize.BANNER)
 // Use test ad unit ID
 adUnitId = "ca-app-pub-3940256099942544/6300978111"
 }
 },
 update = { adView ->
 adView.loadAd(AdRequest.Builder().build())
 }
)
}

```

Поместите AdBanner перед списком JuiceTrackerList в JuiceTrackerApp. Список JuiceTrackerList объявлен в строке 83.

ui/JuiceTrackerApp.kt

```
...
AdBanner(
 Modifier
 .fillMaxWidth()
 .padding(
 top = dimensionResource(R.dimen.padding_medium),
 bottom = dimensionResource(R.dimen.padding_small)
)
)

JuiceTrackerList(
 juices = trackerState,
 onDelete = { juice -> juiceTrackerViewModel.deleteJuice(juice) },
 onUpdate = { juice ->
 juiceTrackerViewModel.updateCurrentJuice(juice)
 scope.launch {
 bottomSheetScaffoldState.bottomSheetState.expand()
 }
 },
)
```

## 6. Получение кода решения

---

Чтобы загрузить код готового codelab, вы можете использовать эти git-команды:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-juice-tracker.git
$ cd basic-android-kotlin-compose-training-juice-tracker
$ git checkout compose-with-views
```

### Заканчиваем!

На этом курс может закончиться, но это только начало вашего путешествия в разработку приложений для Android!

В этом курсе вы научились создавать приложения с помощью Jetpack Compose, современного набора инструментов пользовательского интерфейса для создания нативных приложений для Android. На протяжении всего курса вы создавали приложения со списками, одним или несколькими экранами и перемещались между ними. Вы научились создавать интерактивные приложения, заставляли свое приложение реагировать на ввод пользователя и обновляли пользовательский интерфейс. Вы применяли Material Design и использовали цвета, формы и типографику для оформления приложения.

Вы также использовали Jetpack и другие сторонние библиотеки для планирования задач, получения данных с удаленных серверов, сохранения данных локально и многого другого.

Пройдя этот курс, вы не только получите представление о том, как создавать красивые и отзывчивые приложения с помощью Jetpack Compose, но и будете обладать знаниями и навыками, необходимыми для создания эффективных, удобных в обслуживании и визуально привлекательных приложений для Android. Эта основа поможет вам продолжить изучение и развитие своих навыков в области современной разработки Android и Compose.

Мы хотели бы поблагодарить вас за участие и прохождение этого курса! Мы рекомендуем вам продолжить обучение и расширить свои навыки с помощью дополнительных ресурсов, таких как документация Android Developer, курс Jetpack Compose для Android-разработчиков, Modern Android App Architecture, блог Android Developers, другие codelabs и примеры проектов.

Наконец, не забудьте поделиться тем, что вы создали, в социальных сетях и использовать хэштег #AndroidBasics, чтобы мы и остальные члены сообщества разработчиков Android тоже могли следить за вашим учебным процессом!

Счастливого творчества!!!

<https://developer.android.com/courses/jetpack-compose/course>

<https://developer.android.com/courses/pathways/android-architecture>

<https://medium.com/androiddevelopers>

<https://developer.android.com/get-started/codelabs>